

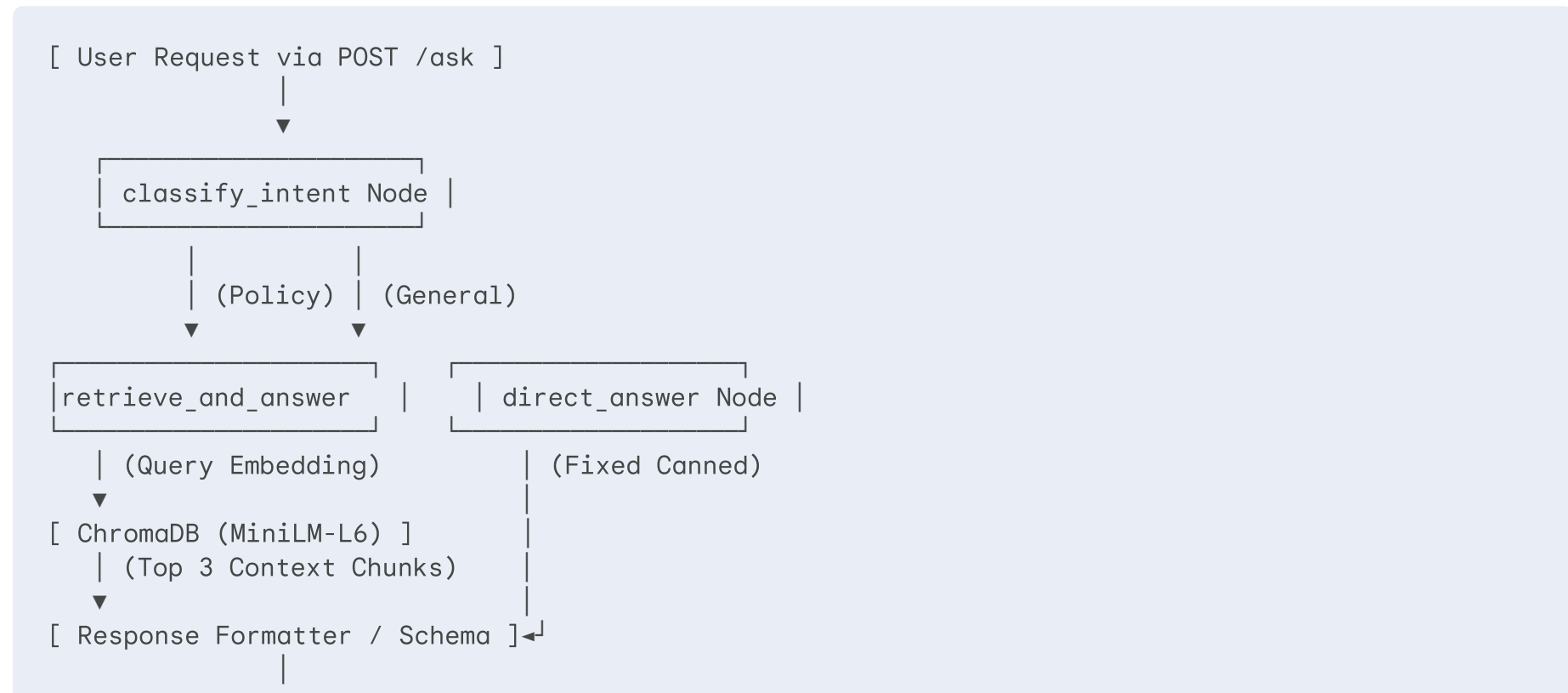
Zepto Support Assistant — Module 3 (/support_assistant)

An enterprise-grade GenAI Customer Support Assistant built for **Zepto**. The system leverages local dense vector retrieval with **ChromaDB** and **sentence-transformers**, orchestrated via a 3-node **LangGraph** state machine.

The service features a **deterministic, fully offline mock mode** (`MOCK_LLM=1`) as the graded baseline, alongside an optional real-LLM extension using Groq (`MOCK_LLM=0`).

1. System Architecture & RAG Pipeline Walkthrough

High-Level Data Flow Diagram



▼
[Pydantic QueryResponse]

Pipeline Stages Breakdown

1. Ingestion Stage (`ingest.py` / `app.py`):

- **Input:** 8 core Zepto policy documents stored in `./docs/` (`doc_01.txt` to `doc_08.txt`).
- **Embedding:** Converts text into 384-dimensional dense vectors locally using `sentence-transformers/all-MiniLM-L6-v2`.
- **Storage:** Persists vectors, IDs (`doc_01` ... `doc_08`), and metadata into a local `ChromaDB` collection named `zepto_policies`.

2. Intent Classification Stage (`graph.py` -> `classify_intent_node`):

- Evaluates the incoming user query.
- **Mock Mode** (`MOCK_LLM=1`): Evaluates a rule-based keyword heuristic. If the query contains keywords like `delivery`, `return`, `refund`, `membership`, `tracking`, `cancel`, `gift card`, or `support hours`, it assigns `policy_question`. Otherwise, it assigns `general_question`. No external LLM calls are made.
- **Real Mode** (`MOCK_LLM=0`): Calls the Groq API to classify the user query.

3. Routing Stage (`graph.py` -> `route_intent`):

- A LangGraph conditional edge routes `policy_question` traffic to `retrieve_and_answer_node` and `general_question` traffic to `direct_answer_node`.

4. Retrieval Stage (`graph.py` -> `retrieve_and_answer_node`):

- **Embedding:** Converts the user's query into a vector using local `all-MiniLM-L6-v2`.
- **Vector Search:** Executes cosine similarity search against `ChromaDB` to fetch the top-3 context chunks.
- **Note:** Retrieval runs locally in both mock and real LLM modes.

5. Generation & Output Schema Stage:

- **Mock Mode (`MOCK_LLM=1`):** For policy queries, formats a deterministic response `Based on the retrieved context: <top chunk excerpt>`, returning retrieved chunk IDs in `sources` and a fixed `confidence` of `1.0`. For general questions, returns a fixed canned message (`I can only answer questions about Zepto policies right now.`).
- **Real Mode (`MOCK_LLM=0`):** Formats `SYSTEM_PROMPT_TEMPLATE` from `prompts.py` (containing strict role, context, negative constraints, and few-shot examples) and prompts the LLM with up to 2 retry attempts on JSON validation failure.

2. Environment Toggles & Configuration

Variable	Values	Default	Purpose
<code>MOCK_LLM</code>	<code>1</code> , <code>0</code>	<code>1</code>	<code>1</code> : Fully offline rule-based graded baseline.
			<code>0</code> : Optional live LLM execution using Groq API.
<code>GROQ_API_KEY</code>	<code>string</code>	<code>""</code>	API key required only when <code>MOCK_LLM=0</code> .
<code>CHROMA_PATH</code>	<code>path</code>	<code>./chroma_db</code>	Storage path for Chroma DB persistent vector database.

3. How to Run

Option A: Running as Unified Script (`app.py`) in Google Colab / Terminal

The entire pipeline (vector store initialization, graph orchestration, and test execution) is available in `app.py` .

```
python app.py
```

Option B: Running FastAPI Application Locally

```
# 1. Install required packages
pip install -r requirements.txt

# 2. Initialize Vector Database
python ingest.py

# 3. Start FastAPI Server
python main.py
# Server will launch at http://localhost:7860
```

Option C: Containerized Run with Docker

```
# Build Docker image
docker build -t zepto-support-assistant .

# Run Docker container locally
docker run -p 7860:7860 zepto-support-assistant

# Test endpoint via curl
curl -X POST http://localhost:7860/ask \
  -H "Content-Type: application/json" \
  -d '{"query": "What is the delivery fee for orders below 149?"}'
```

4. Example API Call Transcripts (MOCK_LLM=1 Baseline)

Below are the raw JSON response transcripts recorded with MOCK_LLM=1 (graded offline baseline):

Call 1: Policy Inquiry (Triggers Retrieval Flow)

Request:

```
POST /ask HTTP/1.1
Host: localhost:7860
Content-Type: application/json

{
  "query": "What is the delivery fee for orders under 149?"
}
```

Response:

```
{
  "answer": "Based on the retrieved context: Zepto delivers grocery and household essentials to serv",
  "sources": [
    "doc_01"
  ],
  "confidence": 1.0
}
```

Call 2: General Inquiry (Triggers Direct Flow)

Request:

```
POST /ask HTTP/1.1
Host: localhost:7860
Content-Type: application/json

{
  "query": "Tell me a software engineering joke."
}
```

Response:

```
{
  "answer": "I can only answer questions about Zepto policies right now.",
  "sources": [],
  "confidence": 1.0
}
```

5. File Manifest

- `docs/doc_01.txt ... docs/doc_08.txt` : Corpus of official Zepto policy files.
- `schemas.py` : Pydantic models for API request validation and response schema.
- `prompts.py` : Structured system prompt template with negative constraints and few-shot examples.
- `ingest.py` : Document loading and ChromaDB vector store builder using `all-MiniLM-L6-v2`.
- `graph.py` : LangGraph state machine with 3 nodes and conditional intent routing.
- `main.py` : FastAPI application exposing `POST /ask`.
- `app.py` : Fully self-contained execution script for single-file deployment or Colab execution.

- `Dockerfile` : Container build configuration exposing port 7860.